PROJECT REPORT



COEP Technological University Pune

Department of Computer Science and Engineering

RISC-V Microarchitecture Simulator: A Configurable Five-Stage Pipelined RV32I Processor with a Multi-Level Cache Hierarchy

Submitted in partial fulfilment of the requirements for Summer Internship/Project 2026

Academic Year: 2026–2027

Submitted by

Abdurrahman Shaikh | MIS: 612415001

Project Guide / Internal Supervisor: Prof. Shirish Purushottam Gosavi

External Guide / Industry Mentor (if applicable): Not Applicable

Contents

Abstract.....4
1. Introduction.....5
1.1 Context.....5
1.2 What this project delivers.....5
1.3 Organisation of this report.....5
2. Problem Statement and Motivation.....6
2.1 Problem statement.....6
2.2 Why existing options were not sufficient.....6
2.3 Motivation.....6
3. Objectives and Scope.....7
3.1 Objectives.....7
3.2 In scope.....7
3.3 Out of scope.....7
4. Background and Literature Review.....8
4.1 The RV32I instruction set.....8
4.2 The classic five-stage pipeline and hazards.....8
4.3 Static branch prediction.....8
4.4 Cache hierarchies.....9
4.5 Related simulators.....9
5. Requirements and Constraints.....10
5.1 Functional requirements.....10
5.2 Non-functional requirements.....10
5.3 Inputs and outputs.....10
5.4 Hardware and software requirements.....11
5.5 Assumptions and constraints.....11
6. System Design and Architecture.....12
6.1 Overall architecture.....12
6.2 Module structure.....12
6.3 Processor datapath.....13
6.4 Pipeline registers.....14
6.5 Control signals.....14
6.6 Memory system and address map.....14
6.7 Cache hierarchy data structures.....15
6.8 Statistics.....15
7. Methodology and Design Decisions.....16
7.1 Reverse-order stage evaluation.....16
7.2 Stalls and flushes.....16
7.3 Forwarding.....17
7.4 Branch prediction and recovery.....17
7.5 Separating function from timing in the memory system.....18
7.6 Structural hazards.....18
7.7 Alternatives considered.....18
8. Implementation.....19
8.1 Build system and toolchain.....19
8.2 Control unit and ALU.....19
8.3 Cache lookup, read and eviction.....19
8.4 Configuration.....20
8.5 Program loaders and the assembler.....20

8.6 Command-line interface....20
8.7 Software support and CoreMark port....21
9. Testing and Validation....22
9.1 Strategy....22
9.2 Workloads....22
9.3 Functional correctness....22
9.4 Retired-instruction count versus Spike....23
9.5 Memory-access counts and cache model versus Spike....23
9.6 Assembly test programs....24
10. Results and Discussion....25
10.1 Experimental setup....25
10.2 Effect of data forwarding....25
10.3 Branch prediction....26
10.4 L1 data-cache size and associativity....26
10.5 Write policy....28
10.6 Depth of the cache hierarchy....28
10.7 Comparison with expected values....29
10.8 CoreMark....30
10.9 Simulation speed....30
10.10 Summary....30
11. Challenges and Limitations....31
11.1 Challenges resolved during development....31
11.2 Remaining limitations....31
12. Conclusion and Future Scope....32
12.1 Conclusion....32
12.2 Future scope....32
Acknowledgments....33
References....34
Appendices....35
Appendix A – Building and running....35
Appendix B – Configuration template (essentials/template.json)....35
Appendix C – Sample statistics report (CoreMark, default configuration)....36
Appendix D – Reproducing the experiments....37
Project Team and Individual Contributions....38
Ethical Practices and Responsible Use....39

Abstract

Computer-architecture concepts such as pipelining, data and control hazards, branch prediction and cache hierarchies are usually taught on paper, which makes their quantitative effect hard to see. This project, the **RISC-V Microarchitecture Simulator**, is a cycle-level model, written from scratch in C, of a five-stage in-order RISC-V processor implementing the complete RV32I base integer instruction set. The simulator models the IF, ID, EX, MA and WB stages with explicit pipeline registers, a forwarding unit with load-use stall detection, a pluggable branch-prediction unit (always-not-taken, always-taken and backward-taken/forward-not-taken), and a configurable timing model of up to three cache levels (split L1 instruction/data caches, unified L2 and L3) in front of DRAM. Cache size, block size, associativity, latency, write policy (write-through/write-back), write-miss policy and replacement policy (FIFO/LRU) are set from a JSON file. Programs are supplied either through a custom two-pass RV32I assembler or as ELF executables produced by the standard RISC-V GCC toolchain, so real C programs and the EEMBC CoreMark benchmark run unmodified apart from a bare-metal port.

The simulator was validated by differential testing against the official RISC-V ISA simulator (Spike) and against natively compiled reference programs. All 216 runs of the six C workloads across the design-space sweep produced bit-exact results. Retired-instruction counts match Spike's exactly (a constant offset of 6 start-up instructions), and direct-mapped L1 data-cache miss rates agree with Spike's independent cache model to within 0.50 percentage points. Experiments show that forwarding reduces CPI by 1.25–2.40×. BTFN static prediction reaches 57–100% accuracy, against 7–54% for always-not-taken. A write-back L1 data cache speeds programs up by as much as 2.7× compared with the default write-through configuration. On CoreMark, the pipeline sustains a CPI of 1.30 with ideal memory and reaches 1.44 CoreMark/MHz.

Keywords: RISC-V, RV32I, pipeline simulator, hazard detection, data forwarding, branch prediction, cache hierarchy, CoreMark, Spike.

1. Introduction

1.1 Context

A processor's performance is determined not only by how many instructions a program executes but by how many clock cycles each instruction costs on average (CPI). In a pipelined processor, CPI is driven by structural, data and control hazards, and by the latency of the memory hierarchy. These effects interact in ways that are difficult to reason about analytically: a branch predictor that removes control stalls exposes more memory stalls, and a larger cache may matter far less than a change of write policy. A simulator that executes real programs and reports *why* every cycle was spent makes these trade-offs visible and measurable.

RISC-V is an open, modular instruction-set architecture whose 32-bit base integer subset, RV32I, contains only about forty instructions while remaining a complete compiler target [1]. This makes it an ideal vehicle for building a processor model from first principles that can still run programs produced by an industrial compiler.

1.2 What this project delivers

The **RISC-V Microarchitecture Simulator** is a cycle-level, execution-driven model of a classic five-stage RV32I pipeline, together with the tooling needed to use it. Its main features are:

- Complete RV32I functional model (R, I, S, B, U and J formats; all loads/stores with alignment and segment checks; `ecall`/`ebreak` for halting).
- Five-stage pipeline (IF, ID, EX, MA, WB) with explicit pipeline-register structures and grouped per-stage control signals.
- Data-hazard handling by forwarding to EX and MA with automatic load-use stalls, or by full interlocking when forwarding is disabled.
- Pluggable branch prediction with ID-stage redirect for predicted-taken branches and flush-on-mispredict recovery.
- Configurable timing model of up to three cache levels plus DRAM, including FIFO/LRU replacement, write-through/write-back, write-allocate/no-write-allocate and structural-hazard detection.
- Two program front-ends: a custom two-pass assembler and an ELF32 loader for binaries built with `riscv64-unknown-elf-gcc -march=rv32i`.
- Observability: a cycle-by-cycle pipeline diagram, register and memory dumps, instruction/data address traces, a leveled log, and 36 statistics counters.
- JSON-driven configuration, so design-space sweeps need no recompilation.

1.3 Organisation of this report

Chapter 2 states the problem and motivation, and Chapter 3 lists the objectives and scope. Chapter 4 reviews the background and related simulators. Chapter 5 sets out the requirements. Chapters 6–8 cover the architecture, the methodology and design decisions, and the implementation. Chapter 9 describes how correctness was verified, including differential testing against Spike. Chapter 10 presents the performance experiments and compares them with published figures. Chapters 11 and 12 discuss limitations and future work. Appendices give build/run instructions, the configuration file, sample output and the reproduction scripts.

2. Problem Statement and Motivation

2.1 Problem statement

To design and implement a configurable, cycle-level simulator of a five-stage pipelined RV32I processor and its memory hierarchy, which executes real compiled programs correctly and reports cycle-accurate statistics that quantify the effect of forwarding, branch prediction and cache organisation on performance.

2.2 Why existing options were not sufficient

Several RISC-V simulators already exist, but each leaves a gap for this purpose:

- **Functional ISA simulators** such as Spike [3] and QEMU are fast and correct but have no notion of a pipeline or of cycles. Spike's optional cache model counts misses but does not translate them into stalls.
- **Educational visual simulators** such as Ripes [5], RARS and Venus show a pipeline graphically but are designed for small hand-written programs. They offer limited or no multi-level cache configuration and are awkward for running a full benchmark and collecting aggregate statistics.
- **Full-system architectural simulators** such as gem5 [4] are extremely capable, but their size (hundreds of thousands of lines) makes them hard to understand end-to-end. Learning *how* a pipeline is built is the point of this project.

2.3 Motivation

Building the machine from scratch forces every architectural rule to become explicit code. Examples are when a value may be forwarded, which pipeline registers a flush clears, and what a dirty eviction costs in a write-back L2. A small, readable C code base (about 6,318 lines excluding the third-party JSON parser) with a configuration file and detailed statistics also serves as a teaching and experimentation platform that others can extend with new predictors, replacement policies or ISA extensions.

3. Objectives and Scope

3.1 Objectives

Table 3.1 lists the objectives of the project, each paired with a concrete check.

Table 3.1: Project objectives and their verification

#	Objective	How it is verified	Status
O1	Execute every RV32I instruction with architecturally correct results.	Program results match native/Spike references (§9.3).	Achieved
O2	Model a five-stage pipeline with explicit pipeline registers and per-cycle visualisation.	Pipeline diagram output; instruction counts match Spike (§9.4).	Achieved
O3	Resolve data hazards by forwarding with load-use stalls, with an option to disable forwarding.	Correct results with forwarding on and off; CPI comparison (§10.2).	Achieved
O4	Support multiple branch-prediction strategies behind a common interface.	NT, T and BTFN implemented; accuracy measured (§10.3).	Achieved
O5	Model a configurable L1-I/L1-D/L2/L3 hierarchy with realistic latencies and policies.	Miss rates validated against Spike's cache model (§9.5, §10.4).	Achieved
O6	Run real C programs and a standard benchmark (CoreMark) compiled by GCC.	ELF loader; CoreMark self-check CRCs pass (§9.3).	Achieved
O7	Make all parameters configurable without recompilation.	JSON configuration used for all 216 experiment runs (§10).	Achieved
O8	Report detailed statistics for analysis.	36 counters printed per run (Appendix C).	Achieved

3.2 In scope

- RV32I user-level integer instructions, a flat physical address space and bare-metal programs.
- Single-issue, in-order, five-stage pipeline; static branch prediction; timing-only cache model.
- A custom assembler, an ELF loader, start-up code (`crt0.S`), a linker script and software helpers for multiply/divide.

3.3 Out of scope

- The M, A, F, D and C extensions, privileged architecture, CSRs, exceptions, interrupts and virtual memory.
- Out-of-order or superscalar execution, multi-core coherence, operating-system support and I/O devices.
- Dynamic branch predictors (2-bit counters, BHT). The interface exists, but the implementations are stubs; see §11.
- Gate-level or RTL timing; the simulator counts cycles, not nanoseconds or power.

4. Background and Literature Review

4.1 The RV32I instruction set

RV32I is the 32-bit base integer ISA of RISC-V [1]. It has 32 general-purpose 32-bit registers (x0 hard-wired to zero), a program counter, and fixed-length 32-bit instructions in six formats. Register specifiers sit in the same bit positions in every format, which simplifies decoding. Immediates are always sign-extended from bit 31.

The formats are summarised in Table 4.1. Because RV32I has no multiply or divide instructions, the C compiler calls library routines (`__mulsi3`, `__divsi3`, …) for these operations. This has a large effect on benchmark results (§10.8).

Table 4.1: RV32I instruction formats

Format	Used by	Immediate
R	Register–register ALU (add, sub, sll, slt, sltu, xor, srl, sra, or, and)	none
I	ALU-immediate, loads (lb, lh, lw, lbu, lhu), jalr, ecall/ebreak	12-bit, sign-extended
S	Stores (sb, sh, sw)	12-bit, split field
B	Conditional branches (beq, bne, blt, bge, bltu, bgeu)	13-bit, multiple of 2
U	lui, auipc	upper 20 bits
J	jal	21-bit, multiple of 2

4.2 The classic five-stage pipeline and hazards

The textbook RISC pipeline [2] divides execution into Instruction Fetch, Instruction Decode/register read, Execute/address calculation, Memory Access and Write Back. Ideally one instruction completes per cycle (CPI = 1). Three kinds of hazard prevent this:

- **Data hazards:** an instruction needs a result that an earlier instruction has not yet written back. Forwarding (bypassing) routes results from later pipeline registers straight to the ALU inputs. Only a load followed immediately by a consumer (a *load-use* hazard) still needs a one-cycle stall.
- **Control hazards:** the address of the next instruction is unknown until a branch is resolved. The pipeline either stalls or speculatively fetches a predicted path and flushes it on a misprediction.
- **Structural hazards:** two stages need the same resource in the same cycle. Here, this happens when the instruction and data sides both access a shared lower-level cache block.

Textbook examples put realistic CPI for such a pipeline at roughly 1.1–1.3 when memory is ideal. A typical breakdown has load-use stalls adding about 0.05 CPI and branch mispredictions about 0.1–0.2 CPI [2], [8], [9].

4.3 Static branch prediction

Static predictors make a fixed guess per branch. *Always-not-taken* simply continues fetching sequentially; it is commonly quoted at about 30–40% accuracy because loop-closing branches are usually taken. *Always-taken* is quoted at about 60–70% [6]. *Backward-taken/forward-not-taken (BTFN)* predicts taken only when the target address is lower than the branch, which captures loops while treating forward if-then branches as not taken [6], [7]. Reported accuracy for such heuristics varies widely by program, from under 10% mispredictions to over 50% [7]. Dynamic predictors (2-bit

saturating counters, branch history tables) learn per-branch behaviour and usually exceed 90% accuracy, but they need extra state.

4.4 Cache hierarchies

A cache is characterised by its capacity, block size and associativity, and by policies for replacement (FIFO, LRU, random), writes (write-through or write-back) and write misses (write-allocate or not). Misses are classified as compulsory, capacity or conflict misses (the "3 Cs"). The average memory access time of a hierarchy is

$$AMAT = t(L1) + m(L1) \times [\, t(L2) + m(L2) \times (\, t(L3) + m(L3) \times t(DRAM) \,) \,]$$

where t is hit time and m is local miss rate. For reference, Hennessy and Patterson's SPEC92 data show miss rates of about 4.6% (direct-mapped) and 3.95% (2-way) for a 4 KB cache, falling to 2.45% and 2.20% at 16 KB [2], [10]. Those numbers are for large benchmark suites; the small kernels used here fit in cache much sooner.

4.5 Related simulators

Table 4.2 positions this work. It sits between functional simulators, which have no timing, and full architectural simulators, which are powerful but opaque. It pairs a real compiler toolchain with a pipeline and multi-level cache model that one person can read in full.

Table 4.2: Comparison with related RISC-V simulators

Simulator	Type	Pipeline timing	Cache model	Runs compiled ELF	Size / approachability
Spike [3]	Functional ISA reference	No	Miss counts only	Yes	Medium; reference quality
QEMU	Dynamic binary translator	No	No	Yes	Large; very fast
gem5 [4]	Full-system architecture simulator	Yes (in-order/O3)	Detailed, coherent	Yes	Very large; steep learning curve
Ripes [5]	Visual educational simulator	Yes (graphical)	Single-level, configurable	Limited	Small programs, GUI-centric
RARS / Venus	Educational assembler + simulator	No	Basic / none	No (assembly)	Small
This work	Cycle-level execution-driven	Yes, 5-stage	L1-I/L1-D/L2/L3, policies	Yes	~6.3 kLOC C, CLI + JSON

5. Requirements and Constraints

5.1 Functional requirements

Table 5.1: Functional requirements

ID	Requirement
FR1	Decode and execute all RV32I computational, control-transfer, load/store and system (`ecall`, `ebreak`) instructions.
FR2	Accept programs as (a) the custom assembler's object listing (`-a`) or (b) a statically linked ELF32 executable (`-e`).
FR3	Advance a global cycle counter and evaluate all five stages every cycle, honouring stalls and flushes.
FR4	Detect RAW hazards; forward when possible, otherwise stall; support forwarding on/off.
FR5	Predict conditional branches with a selectable static strategy; recover from mispredictions.
FR6	Charge every instruction fetch and data access the latency given by the configured cache hierarchy and DRAM.
FR7	Enforce memory-segment protection (no writes to text/rodata, fetches only from text) and natural alignment.
FR8	Halt on `ecall` with a7 = 0 (or on `ebreak`) after draining older instructions.
FR9	Optionally produce pipeline diagrams, register/memory dumps, address traces and logs; always print statistics.
FR10	Read all architectural parameters from a JSON file, falling back to built-in defaults.

5.2 Non-functional requirements

- **Correctness first:** the cache model affects timing only, never data values, so a timing bug cannot corrupt program results (§7.5).
- **Modularity:** each hardware block (stages, forwarding, predictor, caches, memory, loaders) is a separate source file behind a small interface.
- **Portability:** ISO C with the standard library and `libm` only; builds with a single `make` on Linux.
- **Performance:** sufficient to run multi-million-instruction benchmarks in seconds (§10.9).
- **Usability:** a command-line interface with explicit, validated flags and clear fatal-error messages.

5.3 Inputs and outputs

Table 5.2: Simulator inputs and outputs

Kind	Item	Description
Input	Program	ELF32 RISC-V executable (`-e file.elf`) or assembler output (`-a file`)
Input	Configuration	JSON file (`-c config.json`); every key optional
Output	Statistics	Cache hits/misses/write-backs per level, stalls, branches, flushes, forwards, retired instructions and cycles (stdout)
Output	pipevis.txt	Per-cycle contents of IF/ID, ID/EX, EX/MA, MA/WB, WB/IF (`-p`)
Output	register_dump, internal_register_dump	Architectural and pipeline-register state at halt (`-r`)

Kind	Item	Description
Output	memory_dump.bin	Binary image of DRAM at halt (`-m`)
Output	instruction_trace.txt, data_trace.txt	Address traces consumable by the stand-alone `cachesim` tool (`-t`)
Output	log.log	Leveled execution log (`-l`)

5.4 Hardware and software requirements

- Any x86-64 or ARM64 machine running Linux; experiments in this report were run on an Intel Core i5-12450HX laptop.
- GCC (or Clang) and GNU Make to build the simulator; `libm`.
- `riscv64-unknown-elf-gcc` (GCC 13.2 used here) with `-march=rv32i -mabi=ilp32` to compile C programs; optionally Spike for validation.

5.5 Assumptions and constraints

- Programs are bare-metal and statically linked at address 0. The text segment is 0x0–0x1FFFF and read-only data 0x20000–0x3FFFF; data, BSS and heap follow from 0x40000, and the stack grows down from the top of DRAM (1 GB by default).
- Memory is little-endian and byte-addressable; half-word and word accesses must be naturally aligned.
- Every cache line is 4 bytes of bookkeeping granularity, and block sizes are powers of two.
- Write-backs from evictions always succeed (an idealised write buffer).

6. System Design and Architecture

6.1 Overall architecture

Figure 6.1 shows the three phases of using the simulator. In **program preparation**, an assembly file is translated by the built-in two-pass assembler, or a C program is compiled and linked by the RISC-V GCC toolchain with the project's start-up code and linker script. A loader then places the program image into simulated DRAM and sets the PC to the entry point. In **simulation**, the pipeline runs cycle by cycle; every memory access is routed through the cache timing model, which determines how many cycles the access takes. In the **output** phase, the statistics counters and optional dumps and traces are written out.

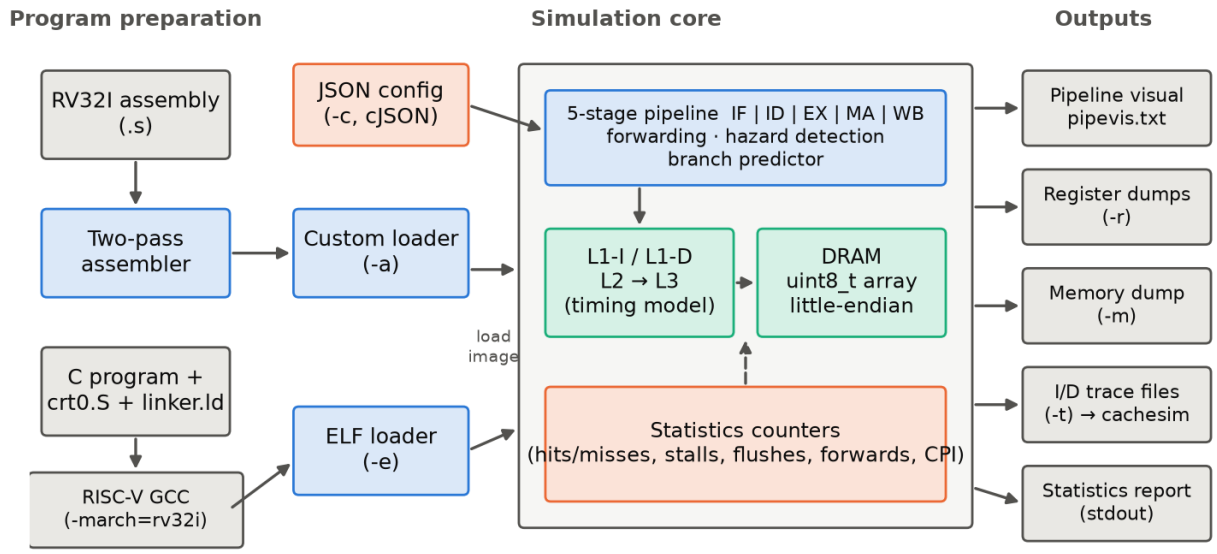


Figure 6.1: Overall software architecture: program preparation, simulation core and outputs

6.2 Module structure

The code is organised so that each hardware block lives in its own module (Table 6.1). This kept the project manageable through about 94 commits between April and October 2026. It also allowed the branch predictor and cache subsystem to be refactored behind stable interfaces.

Table 6.1: Source-tree organisation

Directory / file	Responsibility
src/main.c	Command-line parsing, configuration, loading, run, dumps, statistics
src/execution/simulator.c	The cycle loop: stage ordering, stall/flush/halt handling
src/execution/control.c, alu.c	Instruction table and control-signal generation; ALU, comparators and immediate generator
src/pipeline/{if,id,ex,ma,wb}.c	One file per pipeline stage
src/forwarding/forwarding.c	`forward_ex()` and `forward_ma()` operand bypass and load-use detection
src/branch/*.c	Prediction dispatcher; static (NT, T, BTFN) and dynamic (stub) predictors
src/memory/memory.c	DRAM array, multi-cycle read/write state machines, alignment and segment checks, traces

Directory / file	Responsibility
src/cache/ {cache,read,write,eviction,setters}.c	Cache lookup, per-level read/write paths, FIFO/LRU eviction, bookkeeping bits
src/cache/cachesim.c	Stand-alone trace-driven cache simulator (separate binary)
src/stats/stats.c	Statistics counters and report
src/utils/	ELF and custom loaders, JSON configuration (cJSON), logger, dumps
src/assembler/	Two-pass assembler: parser, syntax/semantic analysis, symbol table, opcode generation
essentials/	crt0.S, linker.ld, malloc/memcpy, libgcc-style mul/div helpers, config template, CoreMark port notes
programs/, tests/	Assembly and C test programs; cache test plan

6.3 Processor datapath

Figure 6.2 gives an abstract view of the datapath; the full gate-level drawing is in the project's draw.io files (`docs/drawings/CPU datapath.drawio`). The stages perform the following work:

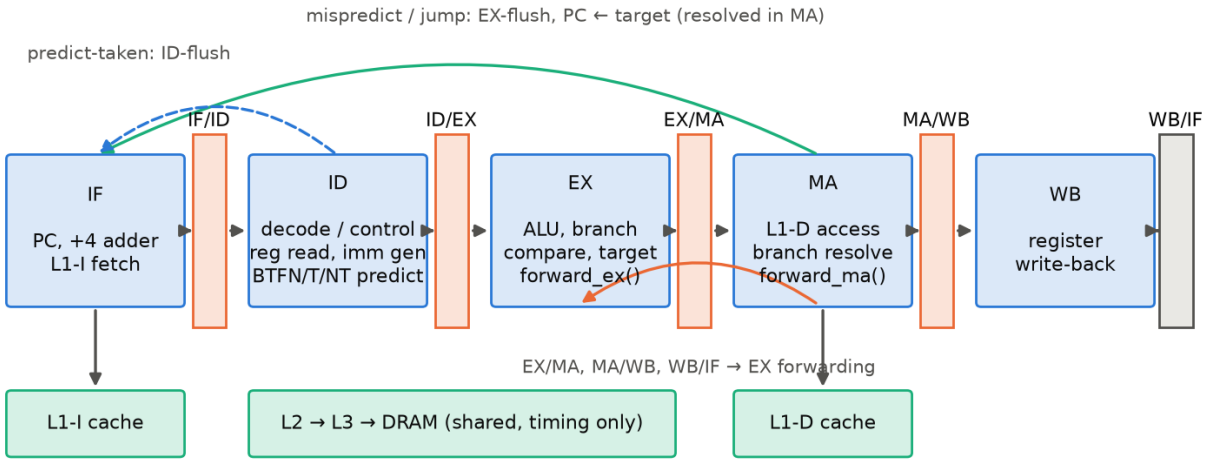


Figure 6.2: Simulated five-stage datapath, pipeline registers, forwarding and redirect paths (abstracted from the project's draw.io datapath)

- **IF** sends the PC to the L1 instruction cache, waits for the access to complete, latches the instruction into IF/ID and increments the PC by 4.
- **ID** extracts the register numbers, calls the control unit to produce grouped control signals, generates the immediate, reads both source registers and queries the branch predictor. When forwarding is disabled, it also runs the interlock check.
- **EX** selects operands (PC or rs1; rs2 or immediate) through the forwarding unit, then performs the arithmetic, logic, set-less-than or branch-compare operation. For control transfers it computes the target.
- **MA** performs loads and stores through the L1 data cache and resolves branches and jumps. On a misprediction or jump it updates the PC and raises a flush.
- **WB** writes the result to the register file (writes to x0 are discarded) and records the last two write-backs for forwarding.

6.4 Pipeline registers

Each pipeline register is a C structure holding the data values *and* the control signals still needed downstream, mirroring how hardware carries control bits alongside the instruction. A printable instruction string travels with each instruction purely for the pipeline diagram.

Listing 6.1 – ID/EX pipeline register (include/pipeline.h)

```
typedef struct ID_EX{
    uint32_t PC, PC_next;          // PC of this instruction and PC + 4
    uint32_t nrs1, R1, nrs2, R2;   // source register numbers and values
    uint32_t imm;                  // generated immediate
    uint32_t nrd;                  // destination register number
    ex_control cs_ex;              // control signals consumed in EX
    ma_control cs_ma;              // ... in MA
    wb_control cs_wb;              // ... in WB
    char * ins;                    // text, for the pipeline diagram only
} ID_EX;
```

Two extra registers have no hardware counterpart. **WB/IF** keeps a copy of the value written back in the current cycle, and **recent_wb** keeps the one from the previous cycle. They exist because the simulator evaluates stages in reverse order (§7.1): by the time EX runs, WB has already consumed MA/WB, so these copies preserve the values the forwarding unit needs.

6.5 Control signals

The control unit decodes the opcode, funct3 and funct7 fields once, in ID, into three structures, one per consuming stage. Grouping signals by stage, rather than keeping one flat set, means each pipeline register copies exactly the groups that are still needed. A bubble is created simply by zeroing a register.

Listing 6.2 – Grouped control signals (include/control.h)

```
typedef struct ex_control{
    uint8_t source1;            // 0: PC, 1: rs1
    uint8_t source2;            // 0: rs2, 1: immediate
    uint8_t sign;               // signed / unsigned operation
    uint8_t type;               // 00 arithmetic, 01 logic, 10 set-less-than, 11 branch/jump
    uint8_t arithmetic_opcode, logic_opcode, branch_opcode;
    uint8_t SYSTEM_CALL;        // 01 ebreak, 10 ecall
} ex_control;
typedef struct ma_control{
    uint8_t mem;                // memory access?
    uint8_t read_write_opcode;  // bit 3: write; bits 2..0: b/h/w/bu/hu
    uint8_t branch_taken;       // bit 0: predicted, bit 1: actual
    uint8_t jump_or_branch;     // 1: jump, 2: conditional branch
} ma_control;
typedef struct wb_control{ uint8_t wb; } wb_control;
```

6.6 Memory system and address map

DRAM is a dynamically allocated, little-endian `uint8_t` array. Its size and the segment limits are configurable (Table 6.2). Loads assemble a word from four bytes and sign- or zero-extend sub-word values. Every access is bounds- and alignment-checked, so a wild pointer in the simulated program stops the simulation with a clear message instead of corrupting the host.

Table 6.2: Default simulated address map (configurable)

Region	Address range	Access
Text (code)	0x0000_0000 – 0x0001_FFFF (128 KB)	Fetch only; writes are fatal
Read-only data	0x0002_0000 – 0x0003_FFFF (128 KB)	Load only
Data, BSS, heap	0x0004_0000 upward	Load/store
Stack	grows down from 0x4000_0000 (top of 1 GB DRAM)	Load/store

6.7 Cache hierarchy data structures

Listing 6.3 – Cache data structures (include/cache.h)

```
typedef struct cache_line{        // one 4-byte word of a block
    uint32_t data, addr;
    uint8_t  valid, dirty;
    uint8_t  done;                 // 0 while a transaction on this block is in flight
    uint64_t reference;            // cycle of fill (FIFO) or last use (LRU)
} cache_line;
typedef struct cache_level{
    int level; cache_line *cache;  // dynamically allocated array
    int read_latency, write_latency;
    int cache_size, block_size, associativity;
    int write_policy;              // 0 write-through, 1 write-back
    int write_miss_policy;         // 0 no-write-allocate, 1 write-allocate
    int replacement_policy;        // 0 FIFO, 1 LRU
} cache_level;
```

Each cache level is a flat array of word-sized `cache_line` entries, laid out set by set; a block of B bytes occupies B/4 consecutive entries. Index and tag are derived from the configured geometry. Four instances exist: `l1_i`, `l1_d`, `l2` and `l3`. `CACHE_LEVELS` selects how many are active; the value −1 selects an ideal single-cycle memory, which is used to isolate pipeline effects.

6.8 Statistics

Counters are incremented at the precise point where an event is decided: hits and misses in the per-level read/write functions, stalls when a memory counter is armed, branch outcomes in MA, flushes in the cycle loop, forwards in the forwarding unit, and retirements in WB. CPI, hit rates and prediction accuracy are derived from them. A complete report is reproduced in Appendix C.

7. Methodology and Design Decisions

7.1 Reverse-order stage evaluation

Real hardware updates all pipeline registers simultaneously on a clock edge. In software this is commonly modelled either by double-buffering every register or by evaluating stages from the back of the pipeline to the front. This simulator uses the second approach: each cycle runs **WB → MA → EX → ID → IF**. Each stage therefore reads its input register before the preceding stage overwrites it, so no second copy is needed. A pleasant side effect is that a register written in WB is visible to ID's register read in the same cycle, which is exactly the "write in the first half, read in the second half" register-file convention of the textbook pipeline [2].

Listing 7.1 – Structure of the cycle loop (simplified from src/execution/simulator.c)

```
while(1){
    wb_stage();
    ma_stage();
    if(mfc == NOT_COMPLETED){ /* D-cache miss: bubble into MA/WB, freeze IF/ID/EX */ cycle++;
continue; }
    else if(FLUSH){ /* jump or mispredict resolved in MA: clear EX/MA, ID/EX, IF/ID */ }
    else if(STRUCTURAL_HAZARD_STALL){ /* retry the conflicting access */ cycle++; continue; }
    ex_stage();
    if(PAUSE || TRAP){ /* ebreak/ecall: drain older instructions, then halt */ }
    else if(FORWARDING_STALL){ /* load-use: bubble into EX/MA */ cycle++; continue; }
    id_stage();
    if(DATA_HAZARD_STALL){ /* interlock when forwarding is off */ cycle++; continue; }
    else if(FLUSH){ /* predicted-taken redirect: clear ID/EX, IF/ID */ }
    if_stage();
    if(mfc_i == NOT_COMPLETED){ /* I-cache miss: bubble into IF/ID */ cycle++; continue; }
    cycle++;
}
```

The cost of this choice is that some values have already moved on by the time a later-evaluated stage looks for them; the extra WB/IF and recent_wb registers (§6.4) compensate for this. Because a stalled stage is simply evaluated again in the next cycle, every stage is written so that it never modifies its own *input* pipeline register; re-executing a stage must always give the same result. For example, EX computes the branch target into a local variable and writes it only to EX/MA, and the store-data operand is forwarded before EX/MA is overwritten.

7.2 Stalls and flushes

Table 7.1 summarises every condition that stops instructions from advancing. The penalties in the last column were measured with micro-benchmarks of 50 repeated instruction pairs under ideal memory, not taken from the design intent. For example, 50 load-use pairs cost exactly 50 extra cycles compared with independent pairs.

Table 7.1: Stall and flush mechanisms (penalties measured with micro-benchmarks)

Event	Detected in	Action	Measured penalty
Load-use dependency	EX (forward_ex)	Bubble into EX/MA; ID/EX held and operands re-read	1 cycle (0 if one independent instruction intervenes)
RAW hazard, forwarding off	ID (set_hazard_stall)	Bubble into ID/EX until the producer has written back	up to 3 cycles

Event	Detected in	Action	Measured penalty
Conditional branch predicted taken	ID	Next cycle: PC ← target, IF/ID cleared	1 cycle
Branch mispredicted / any jump	MA	PC ← correct target; IF/ID, ID/EX, EX/MA cleared; in-flight fetch cancelled	2 cycles
I-cache / D-cache miss	IF / MA	Multi-cycle counter; earlier stages frozen, bubble inserted	latency of the hierarchy
Structural hazard (block busy)	cache lookup	Retry the data access; let IF proceed	≥1 cycle

7.3 Forwarding

When forwarding is enabled, EX asks the forwarding unit for each register operand. The unit searches the in-flight destinations from youngest to oldest, which guarantees the most recent value wins:

Listing 7.2 – EX-stage forwarding (src/forwarding/forwarding.c, comments condensed)

```
uint32_t forward_ex(uint32_t original_value, uint32_t n_reg){
    if(ex_ma.cs_wb.wb && n_reg == ex_ma.nrd && ex_ma.nrd != 0){
        if(ex_ma.cs_ma.mem) FORWARDING_STALL = 1;   // load-use: value not ready yet
        else return ex_ma.result;                   // ALU result one stage ahead
    }else if(ma_wb.cs_wb.wb && n_reg == ma_wb.nrd && ma_wb.nrd != 0){
        FORWARDING_STALL = 1;                       // load finished this very cycle
    }else if(wb_if.cs_wb.wb && n_reg == wb_if.nrd && wb_if.nrd != 0){
        return wb_if.result;                        // value written back this cycle
    }
    return original_value;                          // no hazard
}
```

A second path, `forward_ma()`, supplies the store-data operand in MA so that a store can use a value produced by the instruction directly ahead of it without stalling. When forwarding is disabled, ID instead compares the source registers of the instruction it has just decoded against all three downstream destinations and stalls until none match.

7.4 Branch prediction and recovery

Prediction happens in ID through a single dispatch function. This means a new predictor only has to provide a `predict()` function and, for dynamic schemes, a `store()` function for updates:

Listing 7.3 – Predictor dispatch (src/branch/branch_prediction.c)

```
uint8_t predict(uint32_t ir){
    switch(BRANCH_PREDICTION_STRATEGY){
        case NOT_TAKEN:      return predict_not_taken(ir);
        case TAKEN:          return predict_taken(ir);
        case OFFSET:         return predict_based_on_offset(ir);  // BTFN: taken if imm < 0
        case TWO_BIT_SWITCH: return taken_not_taken_switch(ir);   // stub
        case HISTORY_TABLE:  return branch_history_table(ir);     // stub
        default: log_fatal("Invalid branch prediction strategy."); exit(1);
    }
}
```

The prediction is stored in bit 0 of `cs_ma.branch_taken`, and EX sets bit 1 to the actual outcome. MA compares the two bits. If they differ, MA redirects the PC either to the computed target (actually taken) or to PC + 4 (actually not taken) and raises FLUSH. For a predicted-taken branch, ID sets a one-shot flag; the redirect is applied at the start of the next cycle's ID, so only the single wrong-path fetch is discarded.

7.5 Separating function from timing in the memory system

A key design decision is that **the caches model timing only**. Data is always read from and written to DRAM, which is therefore always coherent. The cache structures track only tags, valid/dirty bits and replacement state, and return the number of cycles an access costs. A bug in the cache model can therefore make statistics wrong but can never make a program compute a wrong answer. This made it possible to develop the complex multi-level write-back logic incrementally. It also means the functional results of §9 validate the pipeline independently of the caches.

Each memory port (instruction and data) is a small state machine with a memory-function-complete flag (`mfc`), an address register (`mar`) and a buffer register (`mbr`). On a new request, the cache model is consulted once and a down-counter is loaded with the access latency. The requesting stage then stalls until the counter reaches zero, at which point the DRAM array is actually read or written.

7.6 Structural hazards

Because L2 and L3 are shared by the instruction and data sides, both can try to use, or evict, the same lower-level block. Each block therefore carries a `done` bit that is cleared while a transaction is outstanding. The replacement routines never evict a block whose `done` bit is clear, and a lookup that finds such a block reports a structural hazard. The cycle loop then retries the data access in the next cycle while still letting the fetch proceed.

7.7 Alternatives considered

Table 7.2: Main design decisions

Decision	Chosen	Alternative	Reason
Language	C	C++ / Python	Direct control of bit-level types (`uint32_t` models modulo-2^{32} arithmetic exactly); speed; closeness to hardware thinking
Register update model	Reverse-order stage evaluation	Double-buffered registers	No duplicate state; natural write-before-read register file
Cache data	Timing-only caches	Caches holding data	Cache bugs cannot corrupt results; simpler write-back logic
Control signals	Per-stage structs	Flat signal set	Bubbles by zeroing; each register carries only what is still needed
Program input	Assembler and ELF loader	Assembler only	ELF enables real C programs and CoreMark
Configuration	JSON at run time	Compile-time macros	Design-space sweeps without rebuilding

8. Implementation

8.1 Build system and toolchain

The project is built with GNU Make into three executables: `assemble` (the assembler), `simulate` (the processor simulator) and `cachesim` (a trace-driven cache simulator sharing the cache sources). Object files go to `build/`. The default flags are `-g -Iinclude`. Target programs are compiled with the RISC-V GCC cross-compiler using the project's start-up code and linker script:

Listing 8.1 – Compiling and running a C program

```
riscv64-unknown-elf-gcc -march=rv32i -mabi=ilp32 -nostdlib -ffreestanding -O2 \
    -T essentials/linker.ld essentials/crt0.S program.c \
    essentials/stdlib/__mulsi3.c essentials/stdlib/__divsi3.c ... -o program.elf
./simulate -e program.elf -c config.json -p -r
```

`crt0.S` sets the stack pointer to `__stack_top`, calls `main`, and then executes `ecall` with a7 = 0, which the simulator treats as the halt system call. The return value of `main` is left in a0, which makes it easy to check results from the register dump.

8.2 Control unit and ALU

`control.c` holds a 41-entry instruction table, shared conceptually with the assembler, and one decoder per format that returns a `cu_output` structure: the immediate format plus the three control groups. The ALU is split into an arithmetic unit (add/sub), a logic unit (and, or, xor, shifts with the shift amount masked to 5 bits as the ISA requires), a less-than comparator and a branch comparator. All values are `uint32_t`; the few signed operations (slt, blt, bge, sra) cast to `int32_t` explicitly, so wrap-around and logical shifts behave exactly as in hardware.

8.3 Cache lookup, read and eviction

`find_block()` computes the set index and tag from the address and configured geometry. It returns 1 on a hit, 0 on a miss when a victim is available, and −1 when every candidate block is busy (structural hazard). The per-level read functions are recursive: a miss in L1-D calls `l2_read()`, which may call `l3_read()`, and so on. Each level adds its latency and fills its own copy on the way back. Eviction scans the set for an invalid block first; otherwise it chooses the block with the smallest `reference` stamp. Under FIFO the stamp is set only on fill, while under LRU `touch()` refreshes it on every hit, so one routine serves both policies. A dirty victim in a write-back level triggers a write into the next level, and the latency of that write is added to the stall.

Listing 8.2 – L1 data-cache read path (src/cache/read.c, simplified)

```
int l1_d_read(uint32_t addr){
    int found = find_block(addr, l1_d);
    if(found == 1){ l1_d_hit_r++; touch(addr, l1_d); set_done(addr, 0, l1_d);
                    return l1_d.read_latency; }
    if(found == -1) return -1;                          // structural hazard
    l1_d_miss_r++;
    int stall = (CACHE_LEVELS > 1) ? l2_read(addr)      // recurse into the next level
                                   : DRAM_READ_LATENCY;
    if(stall == -1) return -1;
    stall += evict(addr, l1_d);                         // + write-back of a dirty victim
    return stall + l1_d.read_latency;
}
```

8.4 Configuration

`configure()` first installs defaults, then overrides any key present in the JSON file (parsed with the third-party cJSON library), and finally allocates DRAM and the cache arrays. The parameters are listed in Table 8.1; the complete template is in Appendix B.

Table 8.1: Configuration parameters and defaults

Parameter	Default	Meaning
CACHE_LEVELS	3	−1 ideal memory, 0 DRAM only, 1–3 number of cache levels
l1_i, l1_d	2 KB, 16 B blocks, 2-way, 1 cycle, write-through, write-allocate, FIFO	Split L1 caches
l2	8 KB, 32 B, 4-way, 10 cycles, write-back, no-write-allocate, FIFO	Unified L2
l3	16 KB, 64 B, 8-way, 30 cycles, write-back, no-write-allocate, FIFO	Unified L3
DRAM_READ/ WRITE_LATENCY	200 / 200	DRAM access time in cycles
DRAM_SIZE	1 GB	Simulated memory size
FORWARDING_SWITCH	1	Enable (1) or disable (0) forwarding
BRANCH_PREDICTION_STRATEGY	0	0 not-taken, 1 taken, 2 BTFN, 3/4 dynamic (stub)
text_segment_limit, rodata_segment_limit	0x1FFFF, 0x3FFFF	Last address of each protected segment

8.5 Program loaders and the assembler

The **ELF loader** validates the ELF32/RISC-V header, copies every `PT_LOAD` segment into DRAM (zero-filling the BSS portion), records the entry point, and labels each instruction slot with its address for the pipeline diagram (the assembler path shows full disassembly). The **assembler** is a classic two-pass design. Pass 1 tokenises each line, checks syntax and operand counts, and records labels in a symbol table together with instruction addresses. Pass 2 resolves label references into PC-relative offsets and emits the 32-bit encodings. It accepts all RV32I mnemonics plus `nop`, register names `r0`–`r31`, decimal or hexadecimal immediates, `offset(reg)` addressing and `//` comments. In verbose mode the output is an annotated listing (index, address, encoding, source) that the custom loader reads.

8.6 Command-line interface

The flags are listed in Table 8.2. Duplicate or conflicting flags, such as supplying both `-a` and `-e`, are rejected with an error message. An excerpt of the pipeline diagram for a short loop is shown below; each column is the instruction held in IF/ID, ID/EX, EX/MA, MA/WB and WB/IF at the end of the cycle, and blank cells are bubbles.

Table 8.2: Simulator command-line flags

Flag	Meaning
-e <file>	Load an ELF executable
-a <file>	Load an assembler listing

Flag	Meaning
-c <file>	JSON configuration
-p	Write the per-cycle pipeline diagram to pipevis.txt
-r	Dump architectural and internal registers
-m	Dump DRAM to memory_dump.bin
-t	Write instruction/data address traces
-l	Enable the execution log

Listing 8.3 – Excerpt of pipevis.txt (always-taken predictor: one bubble per taken loop branch)

```
Cycle 4 : bne r1, r2, loop  | addi r1, r1, 1    | addi r2, r0, 3    | addi r1, r0, 0    |
Cycle 5 : addi r5, r0, 7    | bne r1, r2, loop  | addi r1, r1, 1    | addi r2, r0, 3    |
addi r1, r0, 0
Cycle 6 : addi r1, r1, 1    |                   | bne r1, r2, loop  | addi r1, r1, 1    |
addi r2, r0, 3
Cycle 7 : bne r1, r2, loop  | addi r1, r1, 1    |                   | bne r1, r2, loop  |
addi r1, r1, 1
```

8.7 Software support and CoreMark port

Because RV32I lacks multiply and divide, `essentials/stdlib` provides `__mulsi3`, `__udivsi3`, `__umodsi3`, `__divsi3` and `__modsi3`, together with a bump-pointer `malloc` and `memcpy`. Multiplication is the most frequent of these calls, so `__mulsi3` uses two paths. When the operands are small ($a + b < 511$) it applies the quarter-square identity $a \cdot b = \lfloor (a+b)^2/4 \rfloor - \lfloor (a-b)^2/4 \rfloor$, which needs two lookups in a 1 KB read-only table of 16-bit values. Otherwise it swaps the operands so that the smaller one drives a shift-and-add loop that consumes two multiplier bits per iteration with branch-free additions. The low 32 bits of a product are the same for signed and unsigned operands, so one routine serves both. CoreMark was ported to bare metal by disabling timing, `printf` and floating point in `core_portme`, and it is compiled with `-O2 -DTOTAL_DATA_SIZE=2000`. Since output is unavailable, the experiments in this report use a copy of `core_main.c` changed only to return CoreMark's final CRC and its CRC-error count in a0.

9. Testing and Validation

9.1 Strategy

Correctness was checked at three levels, each with an independent reference:

1. **Hand-written assembly tests** (`programs/assembly`) with documented expected memory contents, covering arithmetic, recursion, sorting, string handling, `jalr` and every load/store width.
2. **Compiled C workloads** whose return value, a checksum, is compared with the same source compiled natively for x86-64.
3. **Differential testing against Spike** [3], the RISC-V reference simulator: the same programs, relinked at Spike's base address with an HTIF exit sequence, give the reference result, the retired-instruction count (from Spike's commit log) and cache statistics (from Spike's `--ic/--dc` model).

Each test was repeated under many configurations (forwarding on/off, every predictor, ideal memory, no caches, 1–3 cache levels, write policies and 72 L1-D geometries per workload). This exercises different stall and flush interleavings.

9.2 Workloads

Table 9.1: Benchmark workloads (compiled with GCC 13.2, -O2, rv32i)

Workload	Description	Character	Instructions
fib(20)	Recursive Fibonacci	Call/return heavy, stack traffic	216,214
quicksort	1,024 pseudo-random words, in-place	Data-dependent branches	143,791
radix sort	1,024 words, 4 passes × 8 bits	Streaming loads/stores, regular loops	135,242
matmul 24²	24×24 integer matrix product	Software multiply (`__mulsi3`) dominated	1,259,334
bubble sort	256 words	Tight nested loops, many stores	202,515
CoreMark	EEMBC CoreMark, 10 iterations, 2 KB data	Industry-standard mix: lists, matrix, state machine, CRC	5,321,340

9.3 Functional correctness

All six workloads produce the reference result (Table 9.2). For CoreMark, the upper 16 bits of a0 hold CoreMark's own count of list, matrix and state CRC mismatches against EEMBC's published values; it is zero, so CoreMark's built-in self-validation also passes. Across the whole experiment matrix, **216 of 216** runs returned the correct value, covering every predictor, forwarding setting and cache configuration. The 11 self-checking assembly programs passed under all seven tested configurations (77 runs).

Table 9.2: Program results under the default configuration (reference: native x86-64 build; Spike for CoreMark)

Workload	Expected a0 (reference)	Simulator a0	Match
fib(20)	6765 (0x1a6d)	6765 (0x1a6d)	Yes
quicksort	1258923079 (0x4b09a447)	1258923079 (0x4b09a447)	Yes
radix sort	1258923079 (0x4b09a447)	1258923079 (0x4b09a447)	Yes

Workload	Expected a0 (reference)	Simulator a0	Match
matmul 24²	1909451840 (0x71cfec40)	1909451840 (0x71cfec40)	Yes
bubble sort	784744565 (0x2ec64075)	784744565 (0x2ec64075)	Yes
CoreMark	64687 (0xfcaf)	64687 (0xfcaf)	Yes

9.4 Retired-instruction count versus Spike

A pipeline that silently drops or duplicates instructions can still produce the right answer by luck, so the number of retired instructions was compared with Spike's commit log (Table 9.3). The difference is exactly **6 for every workload**, including CoreMark's 5.32 million instructions. The six instructions are the difference between the two start-up stubs: the Spike version needs extra instructions for the HTIF `tohost` exit sequence. The simulator therefore retires exactly the same instruction stream as the reference model.

Table 9.3: Dynamic instruction counts

Workload	Spike	This simulator	Difference
fib(20)	216,220	216,214	6
quicksort	143,797	143,791	6
radix sort	135,248	135,242	6
matmul 24²	1,259,340	1,259,334	6
bubble sort	202,521	202,515	6
CoreMark	5,321,346	5,321,340	6

9.5 Memory-access counts and cache model versus Spike

Load and store counts match Spike to within one load (Spike's boot ROM) and two stores (the `tohost` write) (Table 9.4). Spike's cache model was also configured with the same L1-D geometries as the simulator's sweep (§10.4). For **direct-mapped** caches, where the outcome does not depend on the replacement policy, the miss rates agree to within **0.50 percentage points** (mean 0.10 pp) across 24 size/workload combinations. This independently validates the index/tag computation and fill logic. For set-associative caches the mean difference is 0.93 pp, as expected, because Spike uses random replacement while this simulator uses FIFO or LRU.

Table 9.4: Data-memory accesses

Workload	Loads (Spike / sim)	Stores (Spike / sim)
fib(20)	43,170 / 43,169	41,882 / 41,880
quicksort	18,509 / 18,508	8,295 / 8,293
radix sort	19,458 / 19,457	15,363 / 15,361
matmul 24²	35,019 / 35,018	1,745 / 1,743
bubble sort	65,538 / 65,537	31,619 / 31,617
CoreMark	717,808 / 717,807	147,714 / 147,712

9.6 Assembly test programs

The eleven standalone programs in `programs/assembly` store their result at a fixed address; the memory dump (`-m`) was checked against the expected values documented in each file, for example `fact(5) = 120` at 0x900 and the sorted array -5, -2, 0, 3, 7, 9 at 0x800. These programs keep data at 0x800–0xFFF, which is inside the text segment the simulator now protects, so they were run with `text_segment_limit = 0x7FF`. `multiply.s` is a subroutine without a halt and is not a standalone test.

10. Results and Discussion

10.1 Experimental setup

All workloads were compiled once (GCC 13.2.0, `-O2 -march=rv32i -mabi=ilp32`) and run on an Intel Core i5-12450HX under Linux. Unless stated otherwise, the configuration is the default of Table 8.1. "Ideal memory" (`CACHE_LEVELS = −1`) makes every access take one cycle, isolating pipeline effects. A Python driver generated the 216 runs, ran them in parallel and parsed the statistics (Appendix D). Results are deterministic: repeated runs give identical cycle counts.

10.2 Effect of data forwarding

Figure 10.1 and Table 10.1 show that forwarding is the single most effective pipeline mechanism. Without it, every RAW dependency stalls ID for up to three cycles and CPI rises to 1.44–3.02. With forwarding, CPI falls to 1.14–1.66, a **1.26–2.40× speed-up**. Radix sort benefits most because its inner loops form chains of dependent address calculations, loads and increments. The remaining overhead above CPI = 1 comes almost entirely from taken branches and jumps (2 cycles each) and load-use stalls (1 cycle each). This agrees with the textbook five-stage pipeline, whose CPI is typically quoted as 1.1–1.3 [2], [9].

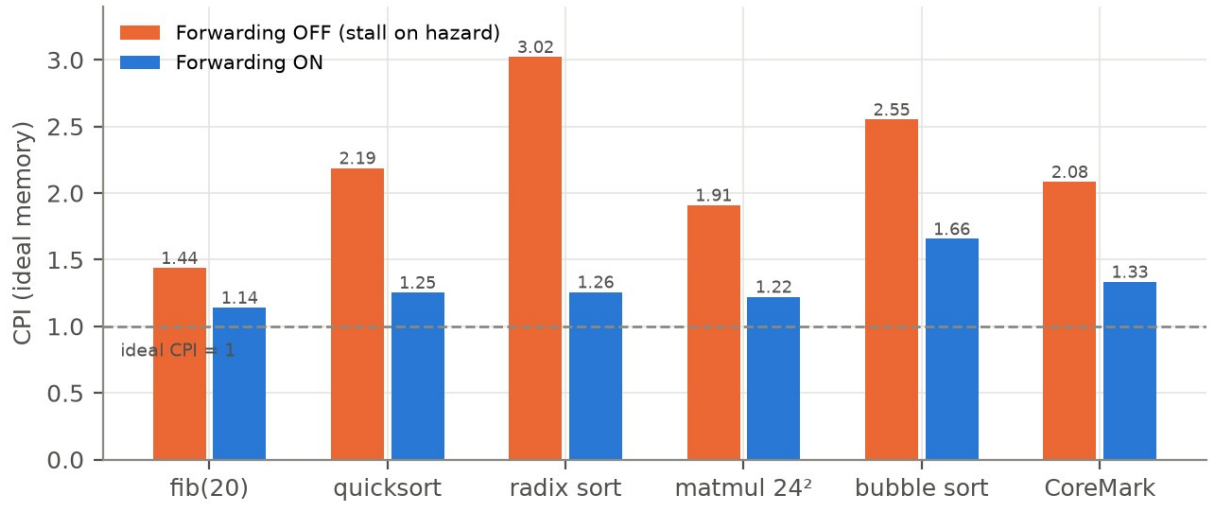


Figure 10.1: CPI with forwarding off and on, under ideal memory

Table 10.1: Forwarding results (ideal memory, not-taken prediction)

Workload	CPI off	CPI on	Speed-up	Forwards to EX	Forwards to MA
fib(20)	1.44	1.14	1.26×	27,172	1,749
quicksort	2.19	1.25	1.75×	59,161	1,024
radix sort	3.02	1.26	2.40×	89,122	9,220
matmul 24²	1.91	1.22	1.56×	381,923	1,728
bubble sort	2.55	1.66	1.54×	105,091	256
CoreMark	2.08	1.33	1.56×	1,828,370	50,301

10.3 Branch prediction

Branches make up 9–33% of executed instructions (Table 10.2). Always-not-taken is correct only 7–54% of the time (46% on CoreMark), and always-taken 46–93%. These figures are in line with the textbook rule of thumb of 30–40% versus 60–70% [6]: loop-closing branches are taken far more often than not. **BTFN is the best or near-best predictor on every workload**, reaching 99.9% on radix sort, 80% on matrix multiply and 73% on CoreMark. It lowers CoreMark's CPI from 1.33 to 1.30. Fibonacci is the exception: its branches are dominated by the forward base-case test, which is taken roughly half the time, so no static scheme does well. Always-taken can be *slower* than not-taken despite higher accuracy (quicksort: 59% vs 41% accuracy, but CPI 1.30 vs 1.25; CoreMark: 54% vs 46%, but CPI 1.40 vs 1.33). Each correctly predicted taken branch still costs a one-cycle redirect bubble, and the forward branches it mispredicts each cost two cycles. This illustrates why accuracy alone does not determine performance.

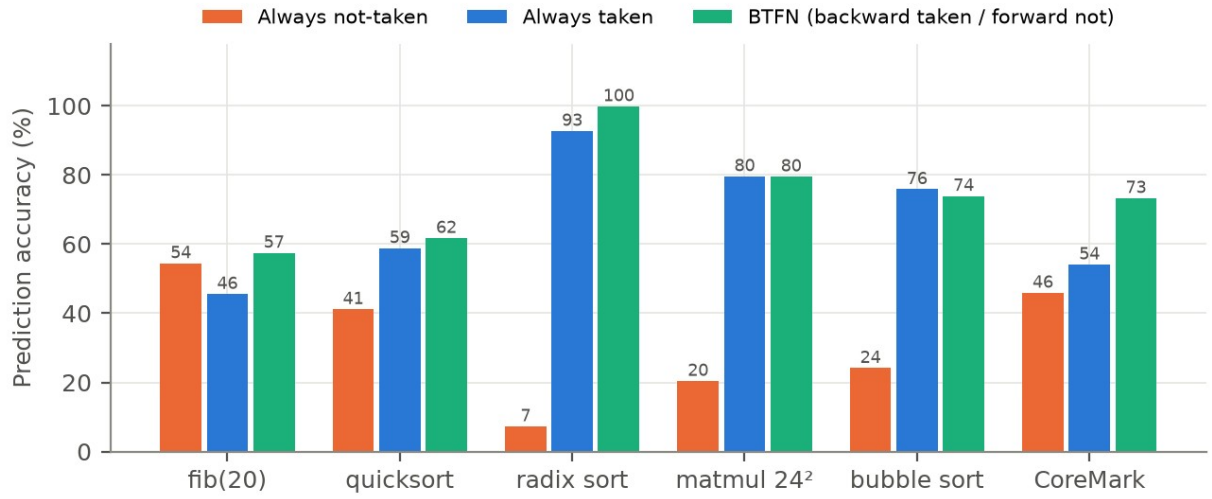


Figure 10.2: Conditional-branch prediction accuracy for the three static predictors

Table 10.2: Branch statistics and CPI by predictor (ideal memory, forwarding on)

Workload	Branches / instr.	Taken	NT acc.	T acc.	BTFN acc.	CPI NT	CPI T	CPI BTFN
fib(20)	8.8%	45.6%	54.4%	45.6%	57.5%	1.14	1.20	1.16
quicksort	18.4%	58.8%	41.2%	58.8%	61.7%	1.25	1.30	1.26
radix sort	10.6%	92.7%	7.3%	92.7%	99.9%	1.26	1.17	1.16
matmul 24²	11.0%	79.6%	20.4%	79.6%	79.5%	1.22	1.18	1.17
bubble sort	32.6%	75.9%	24.1%	75.9%	73.9%	1.66	1.57	1.49
CoreMark	17.1%	54.1%	45.9%	54.1%	73.3%	1.33	1.40	1.30

10.4 L1 data-cache size and associativity

The L1-D was swept from 256 B to 8 KB at 1-, 2- and 4-way associativity with FIFO and LRU replacement (144 runs), with the same geometries simulated in Spike (Figure 10.3, Table 10.3). Three observations stand out:

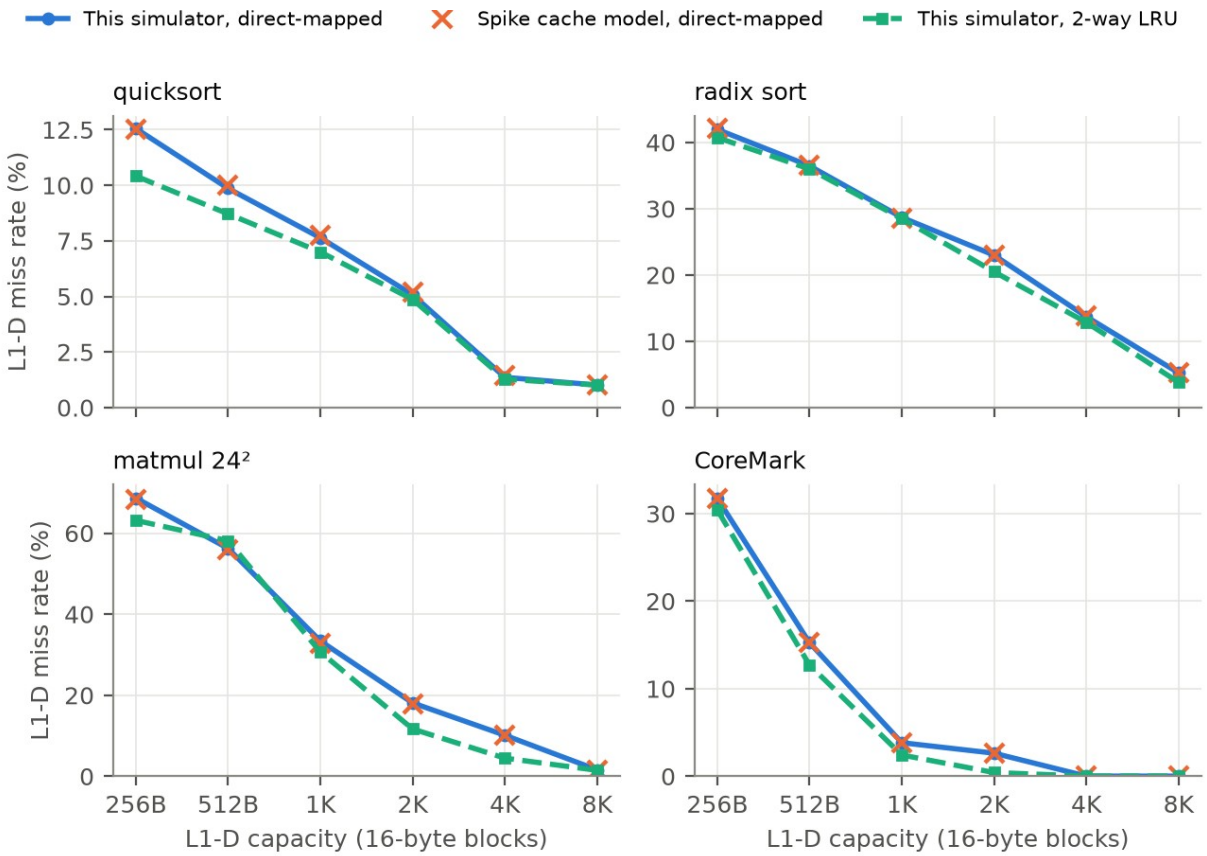


Figure 10.3: L1-D miss rate versus capacity: this simulator compared with Spike's independent cache model

- **Validation:** the direct-mapped curves lie on top of Spike's (maximum deviation 0.50 pp), as discussed in §9.5.
- **Working-set knees:** CoreMark's 2 KB data set, together with the 1 KB quarter-square table used by `__mulsi3`, fits between 2 KB and 4 KB of cache (0.48% at 2 KB 2-way, 0.03% at 4 KB). Quicksort's 4 KB array produces a knee at 4 KB (5.1% → 1.4%). Radix sort streams through two 4 KB arrays and a 1 KB count table, so its miss rate keeps falling until 8 KB. These knees are exactly where the data structures' sizes predict them.
- **Associativity and replacement:** going from direct-mapped to 2-way gives modest gains here, because these kernels have few conflict misses. LRU beat FIFO in 38 of 48 associative configurations, tied in 6 and lost in 4. A notable anomaly is matmul at 512 B, where 2-way (59.0%) is worse than direct-mapped (56.2%). The column-wise walk through B, with a stride of 96 bytes, maps two hot lines into the same set and evicts them cyclically: a classic case where higher associativity with LRU/FIFO does not help a cyclic access pattern larger than the set.

Absolute miss rates at 4–16 KB are lower than Hennessy and Patterson's SPEC92 figures of about 2–5% [10]. This is expected, because these kernels have small working sets, whereas SPEC programs have data sets of many megabytes.

Table 10.3: L1-D miss rate (%) for selected sizes (16-byte blocks, write-back, write-allocate)

Workload	Size	DM sim	DM Spike	2-way FIFO	2-way LRU	2-way Spike (random)	4-way LRU
quicksort	512 B	9.86	10.01	8.82	8.71	9.29	8.69
quicksort	2 KB	5.08	5.19	4.89	4.87	5.03	4.98

Workload	Size	DM sim	DM Spike	2-way FIFO	2-way LRU	2-way Spike (random)	4-way LRU
quicksort	8 KB	1.01	1.02	1.01	1.01	1.05	1.01
radix sort	512 B	36.46	36.52	36.32	35.99	37.06	35.79
radix sort	2 KB	23.00	22.92	22.38	20.50	22.45	20.43
radix sort	8 KB	5.25	5.21	4.42	3.83	5.35	3.75
matmul 24²	512 B	56.22	56.03	58.97	58.12	55.47	57.28
matmul 24²	2 KB	18.12	17.76	13.19	11.69	14.38	12.66
matmul 24²	8 KB	1.61	1.55	1.54	1.49	1.77	1.41
CoreMark	512 B	15.25	15.25	12.67	12.71	12.37	10.77
CoreMark	2 KB	2.64	2.64	0.48	0.43	0.50	0.48
CoreMark	8 KB	0.03	0.03	0.03	0.03	0.03	0.03

10.5 Write policy

In the default configuration the L1-D is write-through *without a write buffer*, so every store waits for the 10-cycle L2. Switching the L1-D to write-back (Table 10.4) removes most write stalls, giving a 1.20× speed-up on CoreMark and **2.65× on fib(20)**, whose register spills to the stack make 19% of instructions stores. Matmul barely changes because it performs few stores. Real write-through caches always pair the policy with a write buffer for this reason [2]; a write buffer is planned in the cache code but not yet implemented (§11).

Table 10.4: L1-D write-through vs. write-back (otherwise default configuration)

Workload	CPI write-through	CPI write-back	Write stalls (WT)	Write stalls (WB)	Speed-up
fib(20)	3.11	1.17	420,561	1,761	2.65×
quicksort	2.07	1.54	103,921	22,671	1.34×
radix sort	3.56	2.82	246,354	118,104	1.26×
matmul 24²	1.30	1.29	51,951	36,201	1.01×
bubble sort	3.25	1.69	321,034	4,864	1.93×
CoreMark	1.66	1.38	1,526,614	19,264	1.20×

10.6 Depth of the cache hierarchy

Without caches, every fetch and data access pays 200 cycles and CPI is 206–296 (Table 10.5, Figure 10.4). A 2 KB L1 alone recovers most of this for compute-bound code (matmul 2.28, CoreMark 7.36) but not for code whose stores go through to DRAM under write-through (fib 40.0). Adding the 8 KB L2 brings every workload within about 2–3× of ideal. The 16 KB L3 matters only when the working set exceeds L2, as with radix sort's two 4 KB arrays (5.30 → 3.56) and CoreMark (1.77 → 1.66). This is the expected diminishing-returns behaviour of a hierarchy, and it can be read directly from the AMAT equation of §4.4.

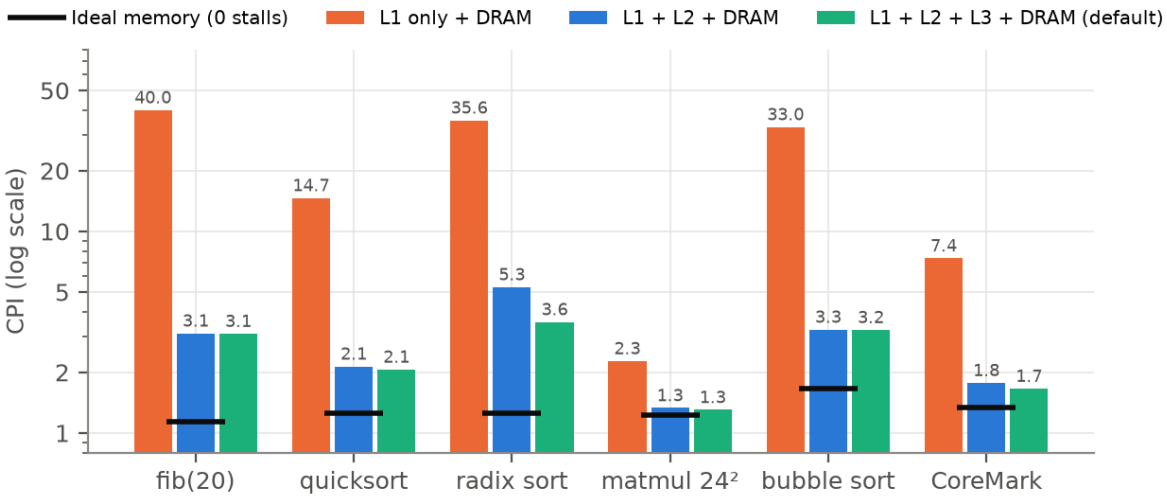


Figure 10.4: CPI as cache levels are added in front of 200-cycle DRAM (log scale)

Table 10.5: CPI versus number of cache levels (default sizes and latencies)

Workload	Ideal	DRAM only	L1	L1+L2	L1+L2+L3
fib(20)	1.14	278.4	39.96	3.12	3.11
quicksort	1.25	237.3	14.66	2.14	2.07
radix sort	1.26	251.4	35.56	5.30	3.56
matmul 24²	1.22	206.0	2.28	1.33	1.30
bubble sort	1.66	296.0	32.96	3.26	3.25
CoreMark	1.33	232.6	7.36	1.77	1.66

10.7 Comparison with expected values

Table 10.6: Measured results against expected values

Metric	Expected (literature)	Measured here	Assessment
Ideal-memory CPI, 5-stage with forwarding	≈1.1–1.3 [2], [9]	1.14–1.66; 1.30 CoreMark + BTFN	Consistent; bubble sort higher due to 33% branches
Load-use penalty	1 cycle [2]	1 cycle (measured)	Matches
Forwarding benefit	CPI 1.5–1.8 → ~1.2 [9]	1.26–2.40× fewer cycles	Consistent
Always-not-taken accuracy	≈30–40% [6]	7–54% (CoreMark 46%)	Consistent
Always-taken accuracy	≈60–70% [6]	46–93% (CoreMark 54%)	Consistent
Direct-mapped L1-D miss rate vs reference model	Spike, same geometry [3]	max |Δ| = 0.50 pp	Matches
Instruction count vs reference model	Spike [3]	identical (+6 start-up)	Matches
CoreMark/MHz, RV32I-class in-order cores	PicoRV32 0.40; NoX (RV32I, 4-stage) 2.50; Ibex-small (RV32IMC) 2.47 [11]	1.13 default; 1.44 ideal + BTFN	Plausible; gap explained in §10.8

10.8 CoreMark

CoreMark/MHz is iterations per million cycles. The simulator achieves **1.13 CoreMark/MHz** in the default configuration and **1.44** with ideal memory and BTFN. The marginal per-iteration rate from a 20-iteration build is 1.45, so start-up overhead is negligible. That is about 3.6× the published 0.40 of the multi-cycle PicoRV32 [11], as expected for a pipelined design. It is below the 2.50 reported for NoX [11], a four-stage RV32I core *also* without hardware multiply. The instruction profile explains most of the gap. CoreMark executes about 530,000 instructions per iteration, and **40.7% of them** are inside `__mulsi3`. CoreMark makes 94,056 multiply calls in 10 iterations, and the quarter-square fast path covers 89% of them, but even this costs about 23 instructions per multiply once the call, return and table lookups are counted. Without the multiply routine, an iteration is about 316,000 instructions. With a single-cycle `mul` instruction and the measured CPI of 1.30, that would be roughly 420,000 cycles per iteration, or about 2.4 CoreMark/MHz, close to the NoX figure. The pipeline itself is not the bottleneck: CPI is 1.30.

10.9 Simulation speed

A cycle-level model is necessarily slower than a functional one (Table 10.7). Still, a few million instructions per second is enough to run CoreMark in seconds. Profiling with `perf` shows that **about 30% of run time** is spent in `log2()` from `libm`: the cache recomputes the number of index and offset bits in floating point on every access. A further ~35% is spent in `get_tag()` and `set_done()`, which recompute the tag of every line in a set on each lookup, and ~13% goes to `log_msg` and `snprintf`, which format the cycle number every cycle even when logging is disabled. Pre-computing the shift amounts once at configuration time, guarding logging behind a cheap level check and building with `-O2` would plausibly give a 2–3× speed-up.

Table 10.7: Host simulation speed (best of 3, Intel i5-12450HX, single thread)

Simulator	Configuration	Time (CoreMark, 5.32 M instr.)	Throughput
This simulator (-g build)	default caches	3.83 s	1.4 MIPS
This simulator (-g build)	ideal memory	1.02 s	5.2 MIPS
This simulator (-O2 build)	default caches	2.45 s	2.2 MIPS
This simulator (-O2 build)	ideal memory	0.70 s	7.7 MIPS
Spike	functional only	0.016 s	≈340 MIPS
Spike	with L1 cache model	0.12 s	≈40 MIPS

10.10 Summary

Every objective of Chapter 3 is met, as the evidence in Chapters 9 and 10 shows. The simulator executes real compiled programs correctly and agrees instruction-for-instruction with the RISC-V reference model. Its cache model reproduces an independent model's miss rates, and its performance trends (forwarding, branch prediction, write policy, hierarchy depth) match those expected from the literature.

11. Challenges and Limitations

11.1 Challenges resolved during development

- **Same-cycle ordering in software.** Evaluating stages in reverse order made results produced in WB and MA vanish from the pipeline registers before EX could forward them. The WB/IF and recent_wb history registers were introduced to fix this.
- **Stale redirect after a flush.** A branch predicted taken could still redirect the PC after a later-detected flush had removed it. The ID stage's pending-redirect flags are now reset on every EX flush (`reset_decode_flags()`).
- **Hazard detection completeness.** An early version stopped checking downstream stages after the first write-back match, and store data (rs2) was not checked when forwarding was off. Both were fixed.
- **Instruction encodings.** Branch/jump offsets and the `jalr` immediate needed different shifting, and the `ebreak` encoding was initially wrong; halting was moved to the standard `ecall` convention.
- **Write-backs during eviction in a shared hierarchy.** Dirty victims in L2/L3 can collide with in-flight accesses. Forced write-backs and the per-block `done` bit (structural-hazard detection) resolve this.
- **Running real software.** Bare-metal start-up code, a linker script matching the protected segments, multiply/divide helpers and a CoreMark port were all needed before the ELF loader was useful.

11.2 Remaining limitations

- **Dynamic predictors are stubs.** `TWO_BIT_SWITCH` and `HISTORY_TABLE` always predict not-taken. The `store()` dispatcher also lacks `break` statements, so it would update both tables once they are implemented.
- `L1_SPLIT` **is parsed but unused:** a unified L1 cannot be selected yet.
- **No write buffer:** write-through levels stall on every store (§10.5).
- **ISA coverage:** RV32I only. There are no M/A/F/C extensions, CSRs, exceptions, interrupts or `fence` semantics, and the only system call is halt, so programs cannot print.
- **Assembler:** no pseudo-instructions apart from `nop` (e.g. `li`, `la`, `mv`), no data directives, and the non-verbose output format cannot be loaded.
- **Testing infrastructure:** the `tests/` directories are planned but empty; validation was performed with external scripts (Appendix D) rather than an automated in-repository suite.
- **Speed:** about 1.4–7.7 MIPS, limited mainly by floating-point `log2()` in the cache path (§10.9). The Makefile also builds with `-g` and no optimisation.

12. Conclusion and Future Scope

12.1 Conclusion

This project built the **RISC-V Microarchitecture Simulator**, a cycle-level model of a five-stage pipelined RV32I processor with forwarding, static branch prediction and a configurable three-level cache hierarchy, together with an assembler, an ELF loader and the bare-metal software support needed to run compiled C programs and CoreMark. Differential testing against the official Spike simulator shows that it executes exactly the same instruction stream with the same results, and that its cache model reproduces Spike's direct-mapped miss rates to within 0.50 percentage points. Across 216 experimental runs it quantified the classic architectural trade-offs. Forwarding cuts cycles by up to 2.40×. BTFN prediction improves accuracy from 46% to 73% on CoreMark. Write-back L1 caching gives up to 2.6× over write-through without a write buffer, and each added cache level shows diminishing returns. All of these agree with textbook expectations.

12.2 Future scope

- **Dynamic branch prediction:** implement the 2-bit saturating counter and a BHT/BTB behind the existing `predict()/store()` interface, and compare them with the static results in Table 10.2.
- **M extension:** hardware multiply/divide would replace the 41% of CoreMark instructions still spent in `__mulsi3` with single `mul` instructions, which by the estimate in §10.8 would raise CoreMark/MHz to about 2.4, and would allow a fair comparison with RV32IM cores.
- **Memory system:** write buffer, unified-L1 option, random and pseudo-LRU replacement, prefetching, and non-blocking caches.
- **Exceptions, interrupts and CSRs** (Zicsr), enabling `printf` through an emulated UART and, eventually, an RTOS.
- **Performance:** pre-compute cache geometry, remove per-cycle string formatting, build with `-O2`.
- **Automated test suite:** turn the assembly programs, C workloads and Spike comparison into an in-repository regression suite run in continuous integration.
- **Visualisation:** plots of CPI stacks and cache heat-maps generated directly from the statistics and traces.

Acknowledgments

I would like to thank Prof. Shirish Purushottam Gosavi, Department of Computer Science and Engineering, COEP Technological University, for guidance throughout this project. I also acknowledge the authors of the RISC-V specifications, the Spike simulator, the GNU RISC-V toolchain, the cJSON library and EEMBC CoreMark, whose open work made this project possible.

References

[1] A. Waterman and K. Asanović, Eds., *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. RISC-V International. [Online]. Available: https://riscv.org/specifications/

[2] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface, RISC-V Edition*. Cambridge, MA, USA: Morgan Kaufmann, 2017.

[3] RISC-V International, "Spike RISC-V ISA Simulator (riscv-isa-sim)," GitHub. [Online]. Available: https://github.com/riscv-software-src/riscv-isa-sim

[4] N. Binkert *et al.*, "The gem5 simulator," *ACM SIGARCH Computer Architecture News*, vol. 39, no. 2, pp. 1–7, 2011.

[5] M. B. Petersen, "Ripes: A visual computer architecture simulator," in *Proc. IEEE/ACM Workshop on Computer Architecture Education (WCAE)*, 2021. [Online]. Available: https://github.com/mortbopet/Ripes

[6] ShanghaiTech University, "CS211 Lecture 11: Speculation (branch prediction)," lecture notes, 2025. [Online]. Available: https://toast-lab.sist.shanghaitech.edu.cn/courses/CS211@ShanghaiTech/Fall-2025/lecture_notes/L11-Speculation.pdf

[7] University of Maryland, Baltimore County, "CMSC 611: Advanced Computer Architecture – Branch prediction," lecture slides. [Online]. Available: https://courses.cs.umbc.edu/graduate/611/Spring00/Slides/3c/cs611.3c.4up.pdf

[8] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.

[9] University of Wisconsin–Madison, "ECE/CS 552: Introduction to superscalar processors," lecture slides. [Online]. Available: https://ece552.ece.wisc.edu/lec09_intro_superscalar.pdf

[10] UMBC, "CMSC 611: Cache performance (miss rate vs. size and associativity, adapted from Hennessy & Patterson)," lecture slides. [Online]. Available: https://userpages.cs.umbc.edu/olano/class/611-08-8/cache2.pdf

[11] A. I. Silva, A. Susin, F. L. Kastensmidt, A. C. S. Beck and J. R. Azambuja, "NoX: A compact open-source RISC-V processor for multi-processor systems-on-chip," arXiv:2406.17878, 2024. (Table III: CoreMark/MHz of NoX, Ibex, PicoRV32 and others.)

[12] EEMBC, "CoreMark," [Online]. Available: https://www.eembc.org/coremark/

[13] D. Gaissmaier, "cJSON: Ultralightweight JSON parser in ANSI C," GitHub. [Online]. Available: https://github.com/DaveGamble/cJSON

[14] RISC-V Collaboration, "riscv-gnu-toolchain," GitHub. [Online]. Available: https://github.com/riscv-collab/riscv-gnu-toolchain

Appendices

Appendix A – Building and running

Listing A.1 – Typical session

```
# build the assembler, simulator and cache simulator
make            # produces ./assemble, ./simulate, ./cachesim

# assemble and run an assembly program (verbose listing is the loadable format)
./assemble -v programs/assembly/factorial_recursive.s prog.obj
./simulate -a prog.obj -c essentials/template.json -p -r -m

# compile and run a C program
riscv64-unknown-elf-gcc -march=rv32i -mabi=ilp32 -nostdlib -ffreestanding -O2 \
    -T essentials/linker.ld essentials/crt0.S programs/c/quick_sort.c \
    essentials/stdlib/__mulsi3.c essentials/stdlib/__udivsi3.c -o quick.elf
./simulate -e quick.elf -r          # result of main() is in r10 (a0)

# replay an address trace through the stand-alone cache model
./simulate -e quick.elf -t && ./cachesim < data_trace.txt
```

Appendix B – Configuration template (essentials/template.json)

Listing B.1 – Default configuration

```
{
    "CACHE_LEVELS": 3,
    "L1_SPLIT": 1,
    "l1_d": {
        "cache_size": 2048,
        "block_size": 16,
        "associativity": 2,
        "read_latency": 1,
        "write_latency": 1,
        "write_miss_policy": 1,
        "replacement_policy": 0,
        "write_policy": 0
    },
    "l1_i": {
        "cache_size": 2048,
        "block_size": 16,
        "associativity": 2,
        "read_latency": 1,
        "write_latency": 1,
        "write_miss_policy": 1,
        "replacement_policy": 0,
        "write_policy": 0
    },
    "l2": {
        "cache_size": 8192,
        "block_size": 32,
        "associativity": 4,
        "read_latency": 10,
        "write_latency": 10,
        "write_miss_policy": 0,
        "replacement_policy": 0,
```

```
        "write_policy": 1
    },
    "l3": {
        "cache_size": 16384,
        "block_size": 64,
        "associativity": 8,
        "read_latency": 30,
        "write_latency": 30,
        "write_miss_policy": 0,
        "replacement_policy": 0,
        "write_policy": 1
    },
    "DRAM_READ_LATENCY": 200,
    "DRAM_WRITE_LATENCY": 200,
    "DRAM_SIZE": 1073741824,
    "FORWARDING_SWITCH": 1,
    "BRANCH_PREDICTION_STRATEGY": 0,
    "text_segment_limit": 131071,
    "rodata_segment_limit": 262143
}
```

Appendix C – Sample statistics report (CoreMark, default configuration)

Listing C.1 – Output of ./simulate -e coremark.elf

```
==================== Simulation Statistics ====================
-------------------- L1 Data Cache --------------------
L1 D Read Hits        : 714292
L1 D Write Hits       : 147088
L1 D Read Misses      : 3515
L1 D Write Misses     : 624
L1 D WriteBacks       : 0
---------------- Instruction Cache --------------------
L1 I Read Hits        : 6799670
L1 I Read Misses      : 10482
L1 I WriteBacks       : 0
---------------------- L2 Cache ----------------------
L2 Read Hits          : 12611
L2 Write Hits         : 146085
L2 Read Misses        : 2010
L2 Write Misses       : 1627
L2 WriteBacks         : 363
---------------------- L3 Cache ----------------------
L3 Read Hits          : 1803
L3 Write Hits         : 1990
L3 Read Misses        : 207
L3 Write Misses       : 0
L3 WriteBacks         : 1
---------------- Memory Requests ---------------------
Data Reads            : 717807
Data Writes           : 147712
Instruction Reads     : 6808223
--------------------- Stall Counts -------------------
Instruction Read Stalls : 183360
Data Read Stalls        : 59310
Data Write Stalls       : 1526614
Structural Hazards      : 0
```

```
-------------------- Branch Stats --------------------
Branch Instructions   : 908362
Branches Taken        : 491345
Branches Not Taken    : 417017
Branch Hits           : 417017
Branch Misses         : 491345
-------------------- Flushes ----------------------
Flushes from ID       : 0
Flushes from EX       : 744869
-------------------- Forwarding -----------------------
Forwards to EX        : 1819040
Forwards to MA        : 86183
---------------- CPU Statistics ----------------------
Instructions Retired  : 5321340
Total Cycles  : 8834371
=========================================================
```

Appendix D – Reproducing the experiments

The experiments were scripted so that every number in this report can be regenerated. `build.sh` compiles each workload three ways: for the simulator (base 0x0, `ecall` halt), for Spike (base 0x8000_0000, HTIF `tohost` exit with a0 as the exit code) and natively for x86-64 (reference checksum). `run.py` writes a JSON configuration per experiment, runs all simulator jobs in parallel in separate directories, parses the statistics and register dump, and runs Spike with the matching `--ic/--dc` geometries and commit logging for instruction counts. `analyze.py` produces the tables and figures. These scripts are supplied alongside this report.

Project Team and Individual Contributions

Team / individual project: Individual project.

Team size: 1.

Role and contributions: I designed and implemented the entire system: the RV32I assembler, the pipeline stages and control unit, the forwarding and hazard-detection logic, the branch-prediction framework, the multi-level cache timing model, the memory system with segment protection, the ELF and custom loaders, JSON configuration, statistics, the bare-metal runtime (start-up code, linker script, arithmetic helpers) and the CoreMark port, together with the design documentation and draw.io datapath diagrams. The repository history records about 94 commits between April and October 2026.

Ethical Practices and Responsible Use

- **Third-party code** is used only where stated and under its licence: cJSON (MIT licence, `src/utils/cJSON.c`) for configuration parsing, and EEMBC CoreMark (CoreMark licence) as a benchmark. CoreMark sources are not redistributed in the repository; the port follows EEMBC's bare-metal porting guide, and results are reported as unofficial because the run rules (≥10 s execution and unmodified reporting) cannot be met on a simulator.
- **Tools:** the GNU RISC-V toolchain and Spike are used as external tools only. Spike served as a validation reference and no Spike code is included.
- **Citations:** all external facts and reference numbers are cited in the References section, and comparison figures from other publications are labelled as such.
- **Data and privacy:** the project processes no personal data. All benchmark inputs are synthetic (pseudo-random arrays, fixed matrices).
- **Honest reporting:** all measurements in this report come from the code in the repository and can be regenerated with the scripts in Appendix D; limitations and unimplemented features are stated explicitly in Chapter 11.
- **Use of AI tools:** Claude Code (Anthropic) was used only to help prepare this report: drafting and formatting the text, scripting the experiment runs and generating the charts. All numbers in the report come from running the project's code.